

باسمه تعالی



فاز اول پروژه اصول طراحی کامپایلر

مدرس: دکتر مهران علیدوست نیا

همکاران: محمد عرفان ربانی ، پرهام محمدخانی ، مهرداد سمیع زاده

پاییز 1404

مقدمه

هدف این پروژه، طراحی و پیاده‌سازی یک زبان برنامه‌نویسی ساده و تولید کد میانی (LLVM IR) با استفاده از زیرساخت LLVM است. این کار شما را با مفاهیم اساسی طراحی کامپایلر و قابلیت‌های کلیدی LLVM آشنا می‌سازد.

ویژگی‌های زبان

• تعریف متغیر

در زبان طراحی شده، متغیرها دارای مقادیری هستند که در زمان compile مشخص خواهد شد. Data type های زبان ما int, float, bool و array می باشند که در ابتدا مقداردهی اولیه ندارند. سینتکس تعریف متغیرها به شکل مقابل می باشد:

```
var x int ;  
var flag bool ;  
var pi float ;  
array arr ;
```

تایپ array مجموعه ای از مقادیر است که میتواند از نوع int یا float یا bool باشد.
مثال مقداردهی اولیه:

```
var x int = 2 ;  
var flag bool = true ;  
var pi float = 3.14;  
array arr = [2, 3, 4];
```

برای نام گذاری متغیر ها تنها مجاز به استفاده از اعداد، حروف و "_" هستیم. همچنین نام متغیر نمی تواند با عدد شروع شود. دقت شود که کلیدواژه های موجود در زبان نمی تواند به عنوان نام یک متغیر به کار روند
مثال هایی از نام گذاری های اشتباه:

var for int ;

var 12a bool ;

var _12a float ;

● انتساب متغیر

در جدول زیر سنتکس های انتساب متغیر آمده که فقط به یک شکل استفاده میشوند در برنامه آمده است .

ADD x y z	$x = y + z$
SUB x y z	$x = y - z$
MUL x y z	$x = y * z$
DIV x y z	$x = y / z$
MOD x y z	$x = y \% z$
INC x	$x++$
DEC x	$x--$
PLE x y	$x = x + y$
MIE x y	$x = x - y$
AND x y z	$x = y \& z$
OR x y z	$x = y z$

محدودیت: دستورات AND/OR فقط برای متغیرهای bool .

از این دستورات نمیتوان در بخش شرط حلقه و دستورات شرطی و همچنین توابع آماده استفاده کرد .

در بخش دستورات شرطی فقط میتوان از دستورات زیر استفاده کرد :

`== , != , > , >= , < , <=`

برای ترکیب چند شرط میتوان از `&&` و `||` استفاده کرد .

● دستورات شرطی

این دستورات شامل شرط و حلقه میباشند. برای سادگی در بدنه آنها مجاز به تعریف متغیر جدید نیستیم، اما میتوانیم آنها را به صورت تودرتو استفاده کنیم .

بلوک شرط با کلیدواژه `if` آغاز میشود. سپس یک عبارت منطقی درون پرانتز نوشته شده و در ادامه آن، بدنه شرط در میان `{ }` قرار میگیرد. در صورتی که حاصل عبارت منطقی `true` باشد، بدنه شرط اجرا میشود و در غیر اینصورت بدنه اجرا نمیشود. میتوان به جای عبارت منطقی از یک متغیر `bool` نیز استفاده کرد.

پس از `if` میتوانیم چندین `if else` و یا یک `else` داشته باشیم .

```
if ( x == 9 ) {  
    print(5) ;  
} else if ( m >= 3 ) {  
    ADD S x m;  
} else {  
    MIE A 1 ;  
}
```

● دستورات حلقه

1. `for`

بلوک حلقه با کلید `for` آغاز میشود و سینتکس داخل آن به شکل زیر خواهد بود .

```
for ( int i = 0; i<10; i++) {  
    INC a ;  
}
```

2. foreach

این حلقه برای پیمایش ارایه عناصر آرایه ها بکار می رود.

```
foreach (num in numbers) {  
    print(num);  
}
```

```
array values = [1, 2, 3, 4, 5];  
foreach (val in values){  
    PLE val 5 ;  
    print(val);  
}
```

● کامنت

در میان دستورات امکان گذاشتن وجود دارد. سینتکس آن به شکل زیر است:

```
/* Comment */
```

```
/* Comment1  
    Comment2 */
```

- **پرینت**

پرینت کردن پاسخ عبارتهای ریاضی، منطقی و متغیرها

```
print(2*5)
```

```
print(s)
```

- **توابع ریاضی**

تبدیل نوع: این تبدیلات ورودی را به نوع مد نظر تبدیل میکنند. و در ادامه برنامه نوع تعیین شده لحاظ میشود

```
to_int(x), to_float(x), to_bool(x)
```

abs: این تابع در ورودی یک مقدار عددی از نوع int یا float میگیرد و در خروجی قدر مطلق آن را از هر تایی که بوده برمیگرداند

```
abs(-5) // return 5
```

- **توابع کار با آرایه**

Length: در ورودی اش یک آرایه می گیرد و طول آن را int برمیگرداند.

```
length(arr)
```

Range Comprehensions: آرایه را ورودی میگیرد و در خروجی یک آرایه با همه عناصر های که شرط گفته شده دارند برمیگرداند .

```
Array evens = [x * 2 for x in nums if x % 2 == 0]
```

Max: آرایه را در ورودی می گیرد و در خروجی کوچکترین عنصر آرایه را برمیگرداند .

```
max(arr)
```

Index: تابع index برای دسترسی به یک عنصر مشخص در یک آرایه استفاده می‌شود. این تابع دو ورودی دریافت می‌کند.

```
index(arr, i)
```

```
print(Index(nums, 5))
```

در صورت وجود نداشتن اندیس با ارور مواجه شویم.

Find : این تابع برای پیدا کردن عنصری درون آرایه با یک شرط خاص استفاده می‌شود. این تابع

دو ورودی دریافت میکند . و در صورت پیدا کردن اولین عنصر مدنظر آن را برمیگرداند . و در صورت پیدا نکردن 1- را برمیگرداند .

```
var element int = find(nums , x > 20)
```

• Pattern Matching :

این روش مشابه switch-case در زبان‌های دیگر است.

```
match x {  
  0 -> print("zero"),  
  1 -> print("one"),  
  _ -> print("other")  
}
```

• Array Iteration :

پیمایش آرایه به ما این امکان را می‌دهد که به تمام عناصر یک آرایه دسترسی داشته باشیم و روی هر عنصر عملیاتی انجام دهیم. این کار معمولاً با استفاده از حلقه‌های for و foreach انجام می‌شود.

```
for (int i=0; i<10; i++){
```

```
INC a[i]
}
```

سیستم مدیریت خطا

ما در این بخش قصد داریم که خطاها را شناسایی کنیم و سعی کنیم به کدنویس در دیباگ کردن کمک کنیم.

الزامات

- تشخیص چندین خطا در هر فایل
- نمایش پیام خطا + شماره خط
- نمایش snippet کد + علامت گذاری محل خطا

مثال خروجی:

```
test1.im:4: error: undefined variable 'x'
      print(x)
        ^
note: declare the variable before use
```

انواع خطاهای الزامی

- خطاهای نحوی: { گمشده، شناسه بد
- خطاهای نحوی: {} گمشده، شناسه بد
- خطاهای نوع: انتساب float به bool
- ارجاع تعریف نشده: استفاده از متغیر قبل از declare
- تقسیم بر صفر

- عملگر یکانی نامعتبر: S از جنس bool است و نمیتوان S++ کرد.

مشاهده پذیری

این بخش به عنوان "چراغ قوه" برای توسعه دهندگان کامپایلر عمل می کند. وقتی در حال پیاده سازی یک کامپایلر هستید، بسیاری از فرآیندها در پشت صحنه اتفاق می افتند و خطاها می توانند بسیار پیچیده و غیرقابل تشخیص باشند.

قابلیت های الزامی که باید انجام دهید :

۱. ردیابی پیشرفته (Advanced Tracing)

- **تعداد نودهای AST:** به شما می گوید ساختار درخت نحو چقدر پیچیده است. اگر تعداد نودها غیرمنتظره زیاد باشد، ممکن است parsing مشکل داشته باشد.

- **Timestampها:** زمان شروع و پایان هر فرآیند را نشان می دهند. این به شما کمک می کند ****bottlenecks** (گلوگاه های عملکردی) را شناسایی کنید.

مثال عملی: اگر تولید کد میانی ۸۰٪ زمان کامپایل را بگیرد، می دانید باید بهینه سازی را روی آن بخش متمرکز کنید.

۲. فایل های واسطه (Intermediate Files)

- **ast.json:** نمایش درخت نحو به صورت JSON. این فایل به شما امکان می دهد ساختار برنامه را به صورت بصری بررسی کنید و مطمئن شوید parsing به درستی انجام شده.
- **ast.json_11، ll.ir:** این فایل های میانی مانند **"عکس های فوری"** از مراحل مختلف کامپایل هستند. اگر در مرحله ای خطا رخ دهد، می توانید به این فایل ها نگاه کنید تا ببینید دقیقاً در کجای فرآیند مشکل به وجود آمده.

۳. نمودارها (CFG/CallGraph)

- **CFG (Control Flow Graph):** نمودار جریان کنترل برنامه را نشان می دهد. این به شما کمک می کند ببینید اجرای برنامه چگونه بین بلوک های کد حرکت می کند.
- **CallGraph:** روابط بین توابع را نمایش می دهد. کدام تابع چه توابعی را فراخوانی می کند.

کاربرد عملی: اگر برنامه به صورت غیرمنتظره ای crash کند، با نگاه کردن به CFG می توانید مسیر اجرایی که منجر به crash شده را دنبال کنید.

۴. خروجی بصری (Visual Output)

- **فایل‌های dot:** این فایل‌ها توسط ابزارهایی مانند GraphViz به تصاویر PNG/SVG تبدیل می‌شوند.
- **مزیت:** دیدن یک **گراف بصری** بسیار ساده‌تر از تحلیل متن خام است. در چند ثانیه می‌توانید ساختار برنامه را درک کنید.

نکات مهم

- از زبان C یا C++ برای توسعه کدها استفاده کنید.
- توسعه هر بخش در فایل‌های جداگانه انجام شود که قابلیت ارزیابی مجزا را داشته باشد.
- کدها خوانایی مناسبی داشته باشند و پیشنهاد می‌شود به درستی کامنت گذاری شود.
- پروژه به صورت تیمی قابل انجام است. اندازه تیم‌ها بین ۲ الی ۳ نفر قابل قبول است و در فاز بعدی، امکان تغییر اعضای تیم وجود ندارد.
- همه اعضای تیم باید در انجام پروژه مشارکت داشته باشند و تسلط هر فرد جداگانه ارزیابی خواهد شد.
- جهت پیاده‌سازی درست و کامل پروژه، پیشنهاد می‌شود اسناد مرتبط با سایت با دقت مطالعه شده و همچنین سه فصل اول کتاب `learn llvm 12` مطالعه شود. کدهای نمونه و روش ارائه مطالب در کتاب به درک شما از پیاده‌سازی کامپایلر مینی‌مال کمک خواهد کرد.
- بخش قابل توجهی از نمره پروژه، مربوط به اجرای صحیح تست‌ها است که در زمان ارائه به شما داده خواهد شد. صحت عملکرد هر بخش از پروژه نیز جداگانه بررسی می‌شود و در صورتی که در بخشی از پروژه اشکال داشته باشید، نمره‌ی باقی قسمت‌ها را دریافت می‌کنید.

موفق باشید